Resource-Aware Data Uploads for Agricultural Robots

Yuanzheng Li, Faaiq Bilal, Natenael Tadele, Jack Baude, Eman Ramadan, Ajay Kumar Gurumadaiah, Zhi-Li Zhang

Application Setting

Agricultural robots generate sensor data for perception and decision-making. Tasks that exceed onboard processing capabilities use remote computation, creating competing demands for uplink connectivity.

Example Applications

- Weed detection, targeted spraying and laser removal
- Disease / Pest detection and treatment
- Selective harvesting · Ripeness assessment
- Precision pruning, thinning, and scouting

- Input modalities are dominated by **Video** and **LiDAR**.
- Video and LiDAR are used for **navigation** and the **target tasks**.
- **Low-power** remote-operated robots can **run longer** as compute is offloaded.

Example Sensor Data Rates

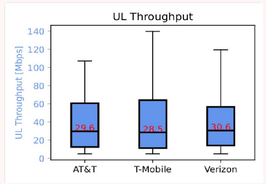
LiDAR: 10-25 Mbps

Video: 5-8 Mbps

- A single application may need multiple video feeds (navigation + task).
- Per-UE **Uplink (UL)** requirements can range from **5-30 Mbps** based on the combination of sensors required.
- **Downlink (DL)** requirements are much lower, mostly containing control commands in the order of 10s of **kbps**.

Wireless Is the Bottleneck

- Cellular Networks provide **low UL throughput** compared to DL throughput.
- Concurrent sensor uploads can exceed the available uplink capacity.
- In areas with weak cellular connectivity, throughput can drop to **< 5 Mbps** without congestion.
- **Which uploads should proceed now, and which can wait, while meeting application requirements?**



Example Tasks







Key Idea: Resource-Aware Admission

Task priority alone is not enough — other work already schedules by priority. The orchestrator is **resource-aware**: it admits high-rate uploads only when the wireless resource can actually carry them.

Task appears → **Server prioritizes** → **UE begins UL transmission** → **Server performs inference** → **Commands sent and executed**

Baseline vs. Proposed

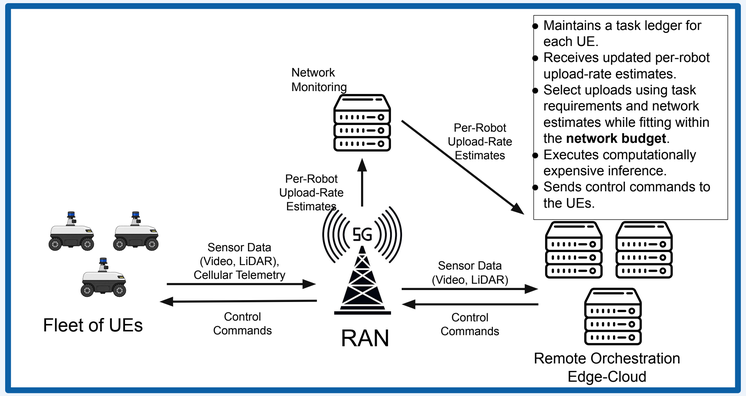
BEST-EFFORT

All robots attempt to transmit sensor data whenever connected.

RESOURCE-AWARE ADMISSION

Robots can transmit data when prompted by a remote server. Otherwise, they retain data locally.

System Architecture



Orchestration Framework

UEs

- Maintain local data buffers and report task requirements and upload progress.
- Transmit sensor data when granted permission.
- Receive processing results and application commands.

Network Monitoring

- Collect upload measurements and available network telemetry.
- Provide per-robot upload-rate estimates to the orchestration server.

Orchestration Edge-Cloud

- Maintains a ledger of tasks ordered by priority.
- Receives updated per-robot upload-rate estimates.
- Selects upload requests to preserve task value delivered at acceptable quality within application delivery windows, while limiting wasted communication.
- Performs expensive inference for tasks, and sends control commands to the UEs.

Success Metrics

- Task value delivered within application requirements.
- Bytes transmitted for expired or canceled requests.
- Upload activity and request waiting time.

Proposed Algorithm

Feasibility-Aware Admission

Runs periodically and on events (new request, rate change, upload done).

1. **Estimate** each robot's current upload rate from recent measurements.
2. **Consider** all unexpired requests, including buffered and paused data. For each task, compute its required bandwidth, value, deadline, and remaining data.
3. **Update value:** $v_i^* = g(v_i, D - d_i, \text{remaining \%})$, a learned function; a task gains value as its deadline approaches.
4. **Select** requests to maximize total value within the expected capacity C:

   $\max \Sigma\, v_i$ over $i \in S$, s.t. $d_i \le D$ for all $i \in S$, $\Sigma\, b_i \le C$

   - admit a request only if remaining data ÷ estimated rate ≤ time left;
   - select the minimum acceptable representation for uploads that have not started;
   - rank requests by value per remaining bit, breaking ties by earlier deadline;
   - pause the lowest-ranked requests;
   - repeat the rate and feasibility checks until the selected set fits the estimated network resources.
5. **Cancel only for a clear gain:** an ongoing task j is canceled only if a new task i has $v_i^* - v_j^* > T$. Paused uploads keep their partial progress.
6. **Grant** uploads to the selected set; other robots retain data locally.

Design Principle

Decide early, give up late.

Screen uploads before transmission begins. Preserve buffered data and partial progress when deferring or pausing a request. Balance the remaining delivery window against the communication required to complete it.

Next Steps

- **Implement** the orchestrator on an edge server with robots on a commercial 5G network: upload permissions, task ledger, and per-robot rate measurement.
- **Validate** on a real testbed with multiple robots uploading at once, comparing against best-effort, priority-only, and deadline-first baselines.
- **Refine** the algorithm with measured data: shared-uplink rate estimation, the value function g, and the threshold T.